

Data Processing for Marine Drifter Trajectory Prediction

ECE/SIOC 228 Final Project

May 16, 2026

1 Raw Data Structure

This section briefly explains the structure of the original NOAA Global Drifter Program 6-hour dataset used in our marine drifter trajectory prediction project. The raw data are organized by year, and each year contains multiple NetCDF files with the extension `.nc`.

1.1 Dataset Organization

The raw dataset is stored in yearly folders:

```
raw_Data/  
  2022/  
  2023/  
  2024/  
  2025/
```

Each yearly folder contains many `.nc` files. The data hierarchy can be understood as:

Dataset $\rightarrow$ Year folder $\rightarrow$ NetCDF files $\rightarrow$ Drifter trajectory $\rightarrow$ 6-hour observations.

Each `.nc` file usually stores one drifter trajectory or one trajectory record. A file does not represent only one 6-hour movement. Instead, it contains many observation points sampled at 6-hour intervals.

For example, if a file has dimensions

$$\text{traj} = 1, \quad \text{obs} = 2036,$$

then it contains one trajectory with 2036 observation points.

1.2 Main Variables in the Raw Files

The main time-series variables used in this project are:

Variable	Meaning	Role
<code>time</code>	Observation time	Core variable
<code>latitude</code>	Drifter latitude	Core variable
<code>longitude</code>	Drifter longitude	Core variable
<code>ve</code>	Eastward velocity component	Core variable
<code>vn</code>	Northward velocity component	Core variable
<code>temp</code>	Sea surface temperature	Optional feature
<code>err_lat, err_lon, err_temp</code>	Error estimates	Optional quality-related features

The most important variables for trajectory prediction are

`time, latitude, longitude, ve, vn.`

They describe when the drifter was observed, where it was located, and how it was moving.

Some files also include metadata variables such as `ID`, `WMO`, `deploy_date`, `start_date`, `end_date`, and `typebuoy`. These variables describe the drifter or trajectory as a whole, but they are not used directly as model input in the current processed dataset.

2 Processed Dataset Structure

This section explains the processed dataset that team members will use for model training. The goal of preprocessing is to convert continuous drifter trajectories into supervised learning samples.

The transformation is:

continuous trajectory $\implies$ historical window $X \rightarrow$ future displacement y .

Each sample contains a short sequence of past drifter states as input and the future displacement as the target.

2.1 Yearly Output Files

Each year is processed separately and saved under:

`processed_Data/<year>_extended/`

For example:

`processed_Data/2022_extended/`
`processed_Data/2023_extended/`
`processed_Data/2024_extended/`
`processed_Data/2025_extended/`

Each folder contains four files:

`drifter_<year>_extended_supervised_windows.npz`
`drifter_<year>_extended_window_metadata.csv`
`drifter_<year>_extended_file_log.csv`
`drifter_<year>_extended_processing_config.json`

Their purposes are:

File	Purpose
<code>supervised_windows.npz</code>	Main arrays for training, validation, and testing
<code>window_metadata.csv</code>	Sample-level information such as source file, time range, current position, future position, and split
<code>file_log.csv</code>	Processing log for each raw NetCDF file
<code>processing_config.json</code>	Processing settings, feature names, split ratios, and summary statistics

For model training, the most important file is the `.npz` file.

2.2 Main Data Arrays

The main training file is:

`drifter_<year>_extended_supervised_windows.npz`

It contains:

Array	Shape	Meaning
<code>X_train</code>	(N_{train}, T, F)	Training input windows
<code>y_train</code>	$(N_{\text{train}}, 2)$	Training targets
<code>X_val</code>	(N_{val}, T, F)	Validation input windows
<code>y_val</code>	$(N_{\text{val}}, 2)$	Validation targets
<code>X_test</code>	(N_{test}, T, F)	Test input windows
<code>y_test</code>	$(N_{\text{test}}, 2)$	Test targets

In the current dataset,

$$T = 8, \quad F = 21.$$

Since each time step represents 6 hours, the model uses

$$8 \times 6 = 48$$

hours of historical information.

The prediction horizon is 4 time steps, which corresponds to

$$4 \times 6 = 24$$

hours into the future.

Therefore, each sample means:

past 48 hours of drifter states $\rightarrow$ latitude/longitude displacement after 24 hours.

The target is not the absolute future position. Instead, it is the future displacement:

$$y_i = \begin{bmatrix} \Delta \text{lat}_i \\ \Delta \text{lon}_i \end{bmatrix} = \begin{bmatrix} \text{lat}_{t+4} - \text{lat}_t \\ \text{lon}_{t+4} - \text{lon}_t \end{bmatrix}.$$

The absolute future position can be recovered by adding the predicted displacement to the current position.

2.3 Feature Description

Each time step contains 21 features. These features include raw position and velocity variables, optional raw variables, constructed physical features, and time encoding features. The constructed physical features have already been computed during data processing and are already included in the 21-dimensional feature vector. Therefore, team members can directly use these features for model training without recomputing them.

The feature names are stored in both the `.npz` file as `feature_cols` and the `processing_config.json` file.

The features are grouped as follows:

Feature Group	Features
Core trajectory features	<code>latitude</code> , <code>longitude</code> , <code>ve</code> , <code>vn</code>
Optional raw features	<code>temp</code> , <code>err_lat</code> , <code>err_lon</code> , <code>err_temp</code>
Constructed physical features	<code>speed</code> , <code>direction</code> , stepwise displacement, velocity change, acceleration, and step distance
Time features	Day-of-year and hour-of-day sine/cosine encodings

The full feature list is:

```
latitude, longitude, ve, vn,
temp, err_lat, err_lon, err_temp,
speed, direction,
delta_lat_step, delta_lon_step,
delta_ve_step, delta_vn_step,
accel_east, accel_north,
step_distance_km,
doy_sin, doy_cos, hour_sin, hour_cos
```

The constructed physical features are not raw variables from the original NetCDF files. They were created during preprocessing from position, velocity, and time information. For example, `speed` and `direction` are computed from `ve` and `vn`; `delta_lat_step` and `delta_lon_step` describe stepwise position changes; `delta_ve_step` and `delta_vn_step` describe stepwise velocity changes; `accel_east` and `accel_north` approximate acceleration; and `step_distance_km` gives the distance traveled between consecutive observations.

Team members may use all 21 features or select a subset depending on the model and experiment design. For example, one model can use only the core features, while another model can use the full extended feature set.

2.4 Standardized and Unstandardized Data

The `.npz` file provides both standardized and unstandardized versions of the input data.

Array Type	Description	Suggested Use
<code>X.train</code> , <code>X.val</code> , <code>X.test</code>	Imputed and standardized	Direct model training
<code>X.train_unscaled</code> , <code>X.val_unscaled</code> , <code>X.test_unscaled</code>	Imputed but not standardized	Feature interpretation and new physical feature construction

For most models, the standardized arrays are recommended as the default input. If team members want to construct additional physical features, they should use the unstandardized arrays because these preserve the original physical units.

After adding new features, the new feature set should be standardized consistently before model training. The scaler should be fitted only on the training set to avoid data leakage.

2.5 Feature Selection and New Physical Features

Team members are free to choose a subset of features for training. This allows comparison between different feature settings, such as:

- core position and velocity features only;
- core features plus speed and direction;

- all 21 extended features;
- customized physics-inspired features.

They may also construct new physical features from the existing variables. Possible examples include average velocity over the history window, net displacement, direction change, acceleration magnitude, estimated advection displacement, and distance from the first point in the input window.

These new features should preferably be constructed from the unstandardized arrays, not from standardized data.

2.6 Train, Validation, and Test Split

The processed dataset is divided into training, validation, and test sets. The current split ratio is:

$$70\% \text{ training, } 15\% \text{ validation, } 15\% \text{ testing.}$$

The training set is used to fit the model. The validation set is used for model selection and hyperparameter tuning. The test set should only be used for final evaluation.

The split information is stored in the `.npz` file and in the `window_metadata.csv` file. The metadata file includes a `split` column for each sample.

2.7 Use with Different Models

For sequence models such as LSTM, GRU, and Transformer, the input can be used directly as:

$$X \in R^{N \times T \times F}.$$

For classical machine learning models such as linear regression or random forest, the input window can be flattened into:

$$X \in R^{N \times (T \cdot F)}.$$

Thus, the same processed dataset can support multiple model types:

Model Type	Suggested Input Shape	Notes
LSTM / GRU	(N, T, F)	Uses temporal order directly
Transformer	(N, T, F)	Uses sequence representation and attention
MLP	$(N, T \cdot F)$ or (N, T, F)	Depends on implementation
Linear Regression / Random Forest	$(N, T \cdot F)$	Usually requires 2D input

2.8 Summary

The processed dataset converts raw 6-hour drifter trajectories into supervised learning samples. Each sample uses the past 48 hours of trajectory information to predict the drifter's latitude and longitude displacement after 24 hours.

The main file for training is the `.npz` file, which contains `X_train`, `y_train`, `X_val`, `y_val`, `X_test`, and `y_test`. The input shape is $(N, 8, 21)$, and the target shape is $(N, 2)$.

The 21 input features include position, velocity, optional raw variables, derived physical features, and time encodings. Team members may use all features, select a subset of features, or construct additional physical features from the unstandardized arrays.